



International
Edition

Problem Solving and Programming Concepts

Ninth Edition

Maureen Sprankle • Jim Hubbard

PEARSON

PROBLEM SOLVING & PROGRAMMING CONCEPTS

This page intentionally left blank

Ninth Edition

PROBLEM SOLVING & PROGRAMMING CONCEPTS

Maureen Sprankle

Professor Emeritus, College of the Redwoods

Jim Hubbard

HGC Technology LLC

Software Consulting

International Edition contributions by

Arindam Chatterjee

Salil Joshi

Prentice Hall

Boston Columbus Indianapolis New York San Francisco Upper Saddle River
Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montreal Toronto
Delhi Mexico City Sao Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo

*To our spouses, Norm and Tamara
and our children, Bonnie, Heidi, Tamara, John and Maliyah
for their continuing support and understanding.*

Editorial Director: Marcia Horton
Editor-in-Chief: Michael Hirsch
Executive Editor: Tracy Dunkelberger
Editorial Assistant: Stephanie Sellinger
Vice President of Marketing: Patrice Jones
Marketing Manager: Yezan Alayan
Marketing Coordinator: Kathryn Ferranti
Vice President, Production: Vince O'Brien
Managing Editor: Jeff Holcomb
Production Project Manager: Kayla Smith-Tarbox
Publisher, International Edition: Angshuman Chakraborty

Acquisitions Editor, International Edition: Arunabha Deb
Publishing Assistant, International Edition: Shokhi Shah
Senior Operations Supervisor: Lisa McDowell
Art Director: Kristine Carney
Cover Designer: Rachael Cronin
Cover Image: © ronfromyork/Shutterstock
Media Editor: Dan Sandin/Wanda Rockwell
Project Manager: Pat Brown
Editorial Production: Sarvesh Mehrotra/Aptara®, Inc.
Cover Printer: Lehigh-Phoenix Color/Hagerstown

Pearson Education Limited
Edinburgh Gate
Harlow
Essex CM20 2JE
England

and Associated Companies throughout the world

Visit us on the World Wide Web at:
www.pearsoninternationaleditions.com

© Pearson Education Limited 2012

The rights of Maureen Sprankle and Jim Hubbard to be identified as authors of this work have been asserted by them in accordance with the Copyright, Designs and Patents Act 1988.

Authorized adaptation from the United States edition, entitled Problem Solving and Programming Concepts, 9th edition, ISBN 978-0-13-249264-5 by Maureen Sprankle and Jim Hubbard published by Pearson Education © 2012.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without either the prior written permission of the publisher or a licence permitting restricted copying in the United Kingdom issued by the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS.

All trademarks used herein are the property of their respective owners. The use of any trademark in this text does not vest in the author or publisher any trademark ownership rights in such trademarks, nor does the use of such trademarks imply any affiliation with or endorsement of this book by such owners.

Microsoft® and Windows® are registered trademarks of the Microsoft Corporation in the U.S.A. and other countries. Screen shots and icons reprinted with permission from the Microsoft Corporation. This book is not sponsored or endorsed by or affiliated with the Microsoft Corporation.

ISBN 10: 0-273-75221-9
ISBN 13: 978-0-273-75221-9

British Library Cataloguing-in-Publication Data
A catalogue record for this book is available from the British Library

10 9 8 7 6 5 4 3 2 1
14 13 12 11 10

Typeset in Times New Roman by Sarvesh Mehrotra/Aptara®, Inc.
Printed and bound by Edwards Brothers in The United States of America

The publisher's policy is to use paper manufactured from sustainable forests.

Contents

Preface 9

UNIT ONE INTRODUCTION TO PROBLEM SOLVING AND PROGRAMMING, 13

Chapter 1 General Problem-Solving Concepts 15

Problem Solving in Everyday Life	15	Summary	19
Types of Problems	17	New Terms	19
Problem Solving with Computers	18	Questions	19
Difficulties with Problem Solving	18	Problems	20

Chapter 2 Beginning Problem-Solving Concepts for the Computer 23

Constants and Variables	25	Expressions and Equations	39
Data Types	28	Summary	46
How the Computer Stores Data	32	New Terms	47
Functions	33	Questions	47
Operators	35	Problems	49

Chapter 3 Planning Your Solution 53

Communicating with the Computer	54	Coding the Solution	73
Organizing the Solution	55	Software Development Cycle	74
Introduction to UML (Unified Modeling Language)	67	Summary	74
		New Terms	75
Using the Tools	71	Questions	75
Testing the Solution	73	Problems	75

UNIT ONE Supplementary Exercises, 77

UNIT TWO LOGIC STRUCTURES, 81

Chapter 4 An Introduction to Programming Structure 83

Pointers for Structuring a Solution	84	Variable Names and the Data Dictionary	97
The Modules and Their Functions	86	The Three Logic Structures	97
Cohesion and Coupling	87	Summary	98
Local and Global Variables	89	New Terms	98
Parameters	91	Questions	99
Return Values	96	Problems	99

Chapter 5 Problem Solving with the Sequential Logic Structure 101

Algorithm Instructions, Flowchart Symbols, and Pseudocode	101	Summary	113
		Questions	114
The Sequential Logic Structure	104	Problems	114
Solution Development	106		

Chapter 6 Problem Solving with Decisions 117

The Decision Logic Structure	118	The Case Logic Structure	147
Multiple <i>If/Then/Else</i> Instructions	120	Codes	149
Using Straight-Through Logic	122	Putting It All Together	150
Using Positive Logic	123	Another Putting It All Together	152
Using Negative Logic	127	Summary	153
Logic Conversion	129	New Terms	154
Which Decision Logic?	132	Questions	154
Decision Tables	132	Problems	155
Putting It All Together	139		

Chapter 7 Problem Solving with Loops 161

The Loop Logic Structure	162	Nested Loops	175
Incrementing	163	Indicators	178
Accumulating	163	Algorithm Instructions and Flowchart Symbols	179
<i>While/WhileEnd</i>	164	Recursion	181
Putting It All Together	166	Summary	181
<i>Repeat/Until</i>	166	New Terms	186
Putting It All Together	169	Questions	186
Automatic-Counter Loop	171	Problems	186
Putting It All Together	175		

UNIT TWO Supplementary Exercises, 189

UNIT THREE DATA STRUCTURES, 191

Chapter 8 Processing Arrays 193

Arrays	194	The Pointer Technique	225
One-Dimensional Arrays	196	Putting It All Together	238
Putting It All Together	201	Summary	247
Two-Dimensional Arrays	203	New Terms	247
Putting It All Together	211	Questions	247
Multidimensional Arrays	220	Problems	248
Table Look-Up Technique	221		

Chapter 9 Sorting, Stacks, and Queues 251

Sorting Techniques	252	New Terms	264
Stacks	259	Questions	264
Queues	260	Problems	265
Summary	264		

Chapter 10 File Concepts 267

Beginning File Concepts	268	Designing Records	271
Records as a Data Structure	268	Summary	275
Primary and Secondary Keys	268	New Terms	275
Algorithm Instructions and Flowchart Symbols	268	Questions	275
Systems Flowcharts	271	Problems	275

Chapter 11 Linked Lists 277

Creating Linked Lists	277	Summary	296
Examples of Adding Data to/Deleting Data from Linked Lists	278	New Terms	296
Algorithms and Flowcharts to Add, Delete, and Access Data in a Linked List	283	Questions	296
		Problems	296

Chapter 12 Binary Trees 299

Creation of Binary Trees	300	New Terms	308
Accessing Data in a Binary Tree	302	Questions	308
Traversal of Binary Trees	306	Problems	308
Summary	308		

UNIT THREE Supplementary Exercises, 309

UNIT FOUR DATABASE MANAGEMENT SYSTEMS, 311

Chapter 13 Database Management Systems 313

Why a DBMS?	314	DBMS Tasks	318
DBMS Components	315	Summary	319
DBMS Models	316	New Terms	320
Client Server Model	317	Questions	320

Chapter 14 Relational Database Management Systems 321

Tables, Records, and Fields	322	Reports	335
Normalizing Tables	323	Planning a Solution Using an RDBMS	335
Entity Relation Model (ERM)	327	Summary	344
Schema	330	New Terms	344
Creating Tables	330	Questions	345
Queries	332	Problems	345
Interface Design	334		

UNIT FIVE OBJECT-ORIENTED PROGRAMMING, 347

Chapter 15 Concepts of Object-Oriented Programming 349

Object-Oriented Programming	350	Summary	363
Graphical User Interface (GUI)	360	New Terms	364
Event-Driven Object-Oriented Programming	360	Questions	364
Interactivity	363	Problems	365

Chapter 16 Object-Oriented Program Design 367

Using UML as a Design Tool	368	New Terms	392
Designing an Object-Oriented Application	374	Questions	392
Interface Design	383	Problems	393
Summary	391		

UNIT SIX INTRODUCTION TO GAME DEVELOPMENT, 395			
Chapter 17 Introduction to Concepts of Game Development Using Object-Oriented Programming 397			
Game Development	398	New Terms	400
Planning the Game	398	Questions	400
Steps to Develop a Simple Game	399	Problems	400
Summary	400		
Chapter 18 Introduction to Assembly Language 403			
Assembly Language Versus		Summary	407
High-Level Languages	404	New Terms	407
Assembly Language Concepts	404	Questions	407
Some Basic Assembly Language Instructions	404	Problems	407
Assembly Language Equivalents to the Four Logic Structures	405		
UNIT SEVEN FILE PROCESSING, 409			
Chapter 19 Sequential-Access File Applications 411			
Processing Sequential-Access Files	413	Error Handling	432
The Primer <i>Read</i>	413	Null Files	434
Designing Output Reports	415	Summary	442
Headings and Line Counters	415	New Terms	443
Control-Breaks	421	Questions	443
Multiple Control-Breaks	425	Problems	443
Using Indicators for Program Control	427		
Chapter 20 Sequential-Access File Updating 445			
Creating Files	446	Putting It All Together	454
The Master File	447	A Useful Alternative Method	464
Transaction Files	447	Summary	469
Activity Files	447	New Terms	469
Backup Files	447	Questions	469
Updating the Master File Using a Transaction File	447	Problems	469
UNIT SEVEN Supplementary Exercises, 471			
APPENDIX A Otto the Robot	473	APPENDIX D Other Problem-Solving Tools	505
APPENDIX B ASCII and EBCDIC Codes for Data Representation	481	APPENDIX E Other Functions	509
APPENDIX C Forms to Use in Problem Solving	485	GLOSSARY	511
		INDEX	519

Preface

Knowledge of problem solving and programming concepts is necessary for those who develop applications for users. Unfortunately, many students have greater difficulty with problem solving than they do with the syntax of computer languages. The art of programming is learning multiple techniques and applying those techniques to specific problems. When students learn basic programming and problem-solving techniques, they can then concentrate on the syntax when learning specific languages. These techniques may be presented in a separate class on problem solving or with a first language course that concentrates on problem solving. This approach tends to decrease students' frustration and improves their success rate.

This book is intended for a one-semester introductory course for programming majors. It can serve as a primary text or as a supplement. Although this book is written for students who have little or no computer experience, those who have studied a computer language can benefit from the generic presentation of the material.

The text provides a step-by-step progression of ideas with detailed explanation and many illustrations, from the basics of mathematical functions and operators to the design and use of techniques such as codes, arrays, pointers, other data structures, database concepts, and object-oriented programming concepts. The text uses problem-solving tools, such as problem analysis charts, interactivity charts, IPO charts, algorithms, and flowcharts and Universal Modeling Language (UML), to design a solution to a problem. The appendices present additional tools, including Nassi-Schneiderman charts, and Warnier-Orr diagrams. Putting It All Together sections illustrate a complete solution for a given problem, using the concepts previously presented. In some cases, an earlier solution is updated to incorporate more sophisticated techniques. Throughout the text, problems presented are typical of the business world and provide excellent experience for students. These problems then can be presented in a language course so that students can finish the solution on the computer.

New in This Edition

The ninth edition responds to suggestions from reviewers and changes in the programming industry. The following changes were made:

- Pseudocode has been added to the algorithm and flowchart figures.
- The Case Structure chapter has been combined with the Decision chapter. The case logic structure is really a type of decision logic structure, therefore the two chapters were combined.
- Updates have been made to the section on cohesion and coupling.
- The Object Oriented Programming (OOP) chapters have been rewritten and updated.
- Universal Modeling Language (UML) is introduced in Chapter 3 and is used as a problem-solving tool in the chapters on OOP.
- A section on the software development life cycle has been added.
- Added ten percent more end of chapter problems.

Organization of the Text

- Unit One, *Introduction to Problem Solving and Programming*, presents basic concepts of problem solving, an introduction to how problems are solved on

computers, mathematical concepts required for problem solving using a computer, and steps for analyzing a problem and designing an appropriate solution.

- Unit Two, *Logic Structures*, presents basic concepts of programming, including local and global variables, parameters, and four basic logic structures. The three basic logic structures are sequential, decision, and loop logic.
- Unit Three, *Data Structures*, presents the concepts of arrays, sorting techniques, stacks, queues, linked lists, and binary trees.
- Unit Four, *Database Management Systems*, presents terminology and techniques to implement an application using a database management system.
- Unit Five, *Object-Oriented Programming*, presents basic concepts in the design of a solution using object-oriented languages. UML is used as the basic problem-solving tool
- Unit Six, *Introduction to Game Development*, presents basic concepts of game development through the use of object-oriented programming. An introduction to assembly language is also presented because many game developers are requiring the knowledge of assembly language to make their games run faster.
- Unit Seven, *File Processing*, presents techniques of file processing. While these chapters are important to those students studying COBOL, the industry is rapidly converting to database management systems. Therefore, these chapters are not as important as they were 10 or 15 years ago.
- The appendices present the Otto the Robot problem, ASCII and EBCDIC codes, blank forms, other problem-solving tools, and some common functions. Otto software is available to instructors via the Pearson Instructor Resource Center. Visit www.pearsoninternationaleditions.com/sprinkle for access.

Resources

Instructor Resources include lecture slides, Instructor's Manual, and Otto software. Instructor Resources are located at www.pearsoninternationaleditions.com/sprinkle. Contact your local Pearson Sales Representative for access information.

Student resources are available at www.pearsoninternationaleditions.com/sprinkle.

Acknowledgments

We are indebted to those who reviewed the manuscript and offered suggestions and constructive comments. In particular, we thank John Arena, *Augusta Technical College Augusta (GA)*; Janet Brown-Sederberg, *Massasoit Community College Brockton (MA)*; Mara Casado, *Manatee Community College (FL)*; Casey Cegielski, *Auburn U Main Campus Auburn (AL)*; Marty Dellinger, *Catawba Valley Community College*; Franklin Fondjo Fotou, *Langston University Langston (OK)*; Roy Foreman, *Purdue University—Calumet*; Mardi Horton, *Cerritos Community College*; Pamela Johnson, *KCTCS-Elizabethtown Elizabethtown (KY)*; Mark Jones, *Lock Haven University Lock Haven (PA)*; Gary Marrer, *Maricopa Community College*; Meg McManus, *Northwest Florida State College Niceville (FL)*; John Mensing, *Brookdale Community College Lincroft (NJ)*; Dennis Roebuck, *Delta College University Center MI (NA)*; Hazem Said, *University of Cincinnati*; Linda Shepherd, *University of Texas-San Antonio San Antonio (TX)*; Joyce Welby, *San Jacinto Coll-South San Jacinto (CA)*. Mark Whigham, *Lawson St-Birmingham Birmingham (AL)*; Melinda White, *Seminole Community College*; and Humayan Zafar, *University of Texas-San Antonio San Antonio (TX)*. Thanks to Tracy Dunkelberger, Pat Brown, and Kayla Smith-Tarbox at Pearson and to Sarvesh Mehrotra at Aptara for their support throughout the production process. The publishers would like to thank Soumen Mukherjee, *RCC Institute of Information Technology* and Arup Kumar Bhattacharjee, *RCC Institute of Information Technology* for reviewing content for the International Edition.

PROBLEM SOLVING & PROGRAMMING CONCEPTS

This page intentionally left blank

UNIT ONE

INTRODUCTION TO PROBLEM SOLVING AND PROGRAMMING

- Chapter 1:** General Problem-Solving Concepts
- Chapter 2:** Beginning Problem-Solving Concepts for the Computer
- Chapter 3:** Planning Your Solution

This page intentionally left blank

Chapter 1

General Problem-Solving Concepts



Overview

Problem Solving in Everyday Life

Types of Problems

Problem Solving with Computers

Difficulties with Problem Solving

Objectives

When you have finished this chapter, you should be able to:

1. Describe the difference between heuristic and algorithmic solutions to problems.
2. List and describe the six problem-solving steps to solve a problem that has an algorithmic solution.
3. Use the six problem-solving steps to solve a problem.

Problem Solving in Everyday Life

People make decisions every day to solve problems that affect their lives. The problems may be as unimportant as what to watch on television or as important as choosing a new profession. If a bad decision is made, time and resources are wasted, so it's important that people know how to make decisions well. There are six steps to follow to ensure the best decision. These six steps in problem solving include the following:

Six Steps of Problem Solving

1. *Identify the problem.* The first step toward solving a problem is to identify the problem. In a classroom situation, most problems have been identified for you and given to you in the form of written assignments or problems out of a book.

However, when you are doing problem solving outside the classroom, you need to make sure you identify the problem before you start solving it. If you don't know what the problem is, you cannot solve it.

2. *Understand the problem.* You must understand what is involved in the problem before you can continue toward the solution. This includes understanding the knowledge base of the person or machine for whom you are solving the problem. If you are setting up a solution for a person, then you must know what that person knows. A different set of instructions might have to be used depending on this knowledge base. For example, you would use a more detailed set of instructions to tell someone how to find a restaurant in your city if he has a limited knowledge of the city than if he knows it well. When you are working with a computer, its knowledge base is the limited instructions the computer can understand in the particular language or application you are using to solve the problem. Knowing the knowledge base is very important since you cannot use any instructions outside this base. You also must know your own knowledge base. You cannot solve a problem if you do not know the subject. For example, to solve a problem involving calculus, you must know calculus; to solve a problem involving accounting, you must know accounting. You must be able to communicate with your client and be able to understand what is involved in solving the problem.
3. *Identify alternative ways to solve the problem.* This list should be as complete as possible. You might want to talk to other people to find other solutions than those you have identified. Alternative solutions must be acceptable ones. You could go from Denver to Los Angeles by way of New York, but this would probably not be an acceptable solution to your travel needs.
4. *Select the best way to solve the problem from the list of alternative solutions.* In this step, you need to identify and evaluate the pros and cons of each possible solution before selecting the best one. In order to do this, you need to select criteria for the evaluation. These criteria will serve as the guidelines for evaluating each solution.
5. *List instructions that enable you to solve the problem using the selected solution.* These numbered, step-by-step instructions must fall within the knowledge base set up in step 2. No instruction can be used unless the individual or the machine can understand it. This can be very limiting, especially when working with computers.
6. *Evaluate the solution.* To evaluate or test a solution means to check its result to see if it is correct, and to see if it satisfies the needs of the person(s) with the problem. (When a person needs a piece of furniture to sleep on, buying her a cot may be a *correct* solution, but it may not be very satisfactory.) If the result is either incorrect or unsatisfactory, then the problem solver must review the list of instructions to see that they are correct or start the process all over again.

If any of these six steps are not completed well, the results may be less than desired.

People solve problems daily at home, or work, or wherever they go. Problems at home include such things as what to cook for dinner, which movie to see this evening, which car to buy, or how to sell the house. At work, the problems might involve dealing with fellow employees, work policies, management, or customers. The better the decisions an employee can make, the more valuable that person will be to the company. In each case, the six steps in problem solving can be followed. Most people use them without even knowing it.

Take the problem of what to do this evening.

1. Identify the problem. How do the individuals wish to spend the evening?
2. Understand the problem. With this simple problem, also, the knowledge base of the participants must be considered. The only solutions that should be selected are ones that everyone involved would know how to do. You probably would not select as a possible solution playing a game of chess if the participants did not know how to play.
3. Identify alternatives.
 - a. Watch television.
 - b. Invite friends over.
 - c. Play video games.
 - d. Go to the movies.
 - e. Play miniature golf.
 - f. Go to the amusement park.
 - g. Go to a friend's party.The list is complete only when you can think of no more alternatives.
4. Select the best way to solve the problem.
 - a. Weed out alternatives that are not acceptable, such as those that cost too much money or do not interest one of the individuals involved.
 - b. Specify the pros and cons of each remaining alternative.
 - c. Weigh the pros and cons to make the final decision. This solution will be the best alternative if all the other steps were completed well.
5. Prepare a list of steps (instructions) that will result in a fun evening.
6. Evaluate the solution. Are we having fun yet? If nobody is having fun, then the planner needs to review the steps to have a fun evening to see whether anything can be changed, if not then the process must start again.

By going through these steps, the problem solver can be assured that he has arrived at the best possible solution and will achieve the desired results.

Types of Problems

algorithmic solution

algorithm

heuristic solution

Problems do not always have straightforward solutions. Some problems, such as balancing a checkbook or baking a cake, can be solved with a series of actions. These solutions are called **algorithmic solutions**. Once the alternatives have been eliminated, for example, and once one has chosen the best among several methods of balancing the checkbook, the solution can be reached by completing the actions in steps. These steps are called the **algorithm**. The solutions of other problems, such as how to buy the best stock or whether to expand the company, are not so straightforward. These solutions require reasoning built on knowledge and experience, and a process of trial and error. Solutions that cannot be reached through a direct set of steps are called **heuristic solutions**.

The problem solver can use the six steps for both algorithmic and heuristic solutions. However, in step 6, evaluating the solution, the correctness and appropriateness of heuristic solutions are far less certain. It's easy to tell if your completed checkbook balance is correct and satisfactory, but it's hard to tell if you have bought the best stock. With heuristic solutions, the problem solver will often need to follow the six steps more than once, carefully evaluating each possible solution before deciding which is best.

Furthermore, this same solution may not be correct and satisfactory at another time, so the problem solver may have to reevaluate and resolve the same problem later. The stock that did well in January may do poorly in June.

Most problems require a combination of the two kinds of solutions.

Problem Solving with Computers

solution
results
program

In this book, the term **solution** means the instructions listed during step 5 of problem solving—the instructions that must be followed to produce the best results. **Results** means the outcome or the completed computer-assisted answer. **Program** means the set of instructions that make up the solution after they have been coded into a particular computer language.

Computers are built to deal with algorithmic solutions, which are often difficult or very time consuming for humans. People are better than computers at developing heuristic solutions. Solving a complicated calculus problem or alphabetizing 10,000 names is an easy task for the computer, but the problem of how to throw a ball or how to speak English is not. The difficulty lies in the programming. How can problems such as how to throw a ball or speak English be solved in a set of steps that the computer can understand?

The field of computers that deals with heuristic types of problems is called artificial intelligence. Artificial intelligence enables a computer to do things like build its own knowledge bank and speak in a human language. As a result, the computer's problem-solving abilities are similar to those of a human being. Artificial intelligence is an expanding computer field, especially with the increased use of Robotics.

Until computers can be built to think like humans, people will process most heuristic solutions and computers will process many algorithmic solutions. Therefore, this book will deal only with algorithmic solutions. Heuristic problem solving can help determine alternative solutions. However, for computer use, they must be transformed into an algorithmic format.

Difficulties with Problem Solving

People have many problems with problem solving. Some have not been taught how to solve problems. Others are afraid to make a decision for fear it will be the wrong one. Often, when people go through the problem-solving process, they complete one or more of the steps inadequately. They may not define the problem correctly or may not generate a sufficient list of alternatives. When choosing the best alternative, they may eliminate good alternatives or list the pros and cons too hastily. They may not use a logical sequence of steps in their solution, or they may focus on details before the framework for the solution is in place. Finally, they may incorrectly or haphazardly evaluate the solution.

The problem-solving process is not easy. It takes practice and time to perfect, but in the long run the process proves to be of great benefit.

When solving problems on the computer, one of the most difficult tasks for the problem solver is writing the instructions. Take the task of deciding which number is the largest from a group of three numbers. Almost anyone can immediately tell which is the largest, but many cannot explain the steps they followed to arrive at it. Most

people will say, “I can’t explain how I know, I just know it!” This explanation is not good enough for the computer. The computer is a tool that will perform only tasks that the user can explain.

The computer has a specific system of communication that programmers and users must learn. This system demands that no step in the solution to a problem be left unstated and that all steps be in the proper order. You must assume the computer knows nothing except what you tell it and think of it as an ignorant but efficient aid to problem solving.

Summary

The six steps in problem solving lead to the best possible solution to a problem. They include (1) identifying the problem, (2) understanding the problem, (3) identifying alternative solutions, (4) selecting the best solution, (5) preparing a list of instructions to solve the problem by the chosen solution, and (6) evaluating the solution. Solutions to problems are classified as algorithmic or heuristic. Algorithmic solutions are reached in a series of steps. Heuristic solutions are attained through trial and error. Algorithmic solutions are easier to define for computer use than heuristic ones.

Several things can go wrong in the problem-solving process. Sometimes the problem solver does not define the problem correctly or does not complete one or more of the steps. At other times, the list of alternatives is not sufficient or the list of instructions to the computer is incorrect. Sometimes one gets lost in details and overlooks the general framework, or does not test the solution. If the problem-solving process is incomplete, the solution will not produce the desired results.

The steps in problem solving can be applied to problems in daily life as well as to problems put on the computer. Good problem-solving techniques enable you to look at a problem logically and unemotionally, saving time and other resources.

New Terms

algorithm	heuristic solution	results
algorithmic solution	program	solution

Questions

1. What are the different types of problems?
2. Explain the concept of an algorithm.
3. Name three current problems in your life that could be solved through an algorithmic process. Explain why each of these problems is algorithmic in nature.
4. What is a heuristic solution to a problem?
5. Name three current problems in your life that might be solved through a heuristic approach. Explain why each of these problems is heuristic in nature.
6. Name three problems that might arise at home, at school, or in a business that could be solved more efficiently with computer assistance. Do these problems require an algorithmic or heuristic solution? Why?
7. State a reason why each of the six problem-solving steps is important in developing the best solution for a problem. Give one reason for each step.

Problems

1. Complete the six problem-solving steps to solve one of the problems you listed in question 3. Follow the form outlined as follows:

Step 1: Identify the problem.

Step 2: Understand the problem.

- a. Comments about the problem to aid in understanding it.
- b. Description of the knowledge base (this list would include what you would be expected to know to follow the solution).

Step 3: Identify alternative solutions.

Solution	Pros	Cons
----------	------	------

a.

b.

c.

.

.

.

Step 4: Select the best solution.

Why did you select this solution?

Step 5: List a set of numbered step-by-step instructions to attain the solution.

1.

2.

3.

.

.

.

Step 6: Test the solution.

Does this solution work?

If not, how might you change the solution so it will work?

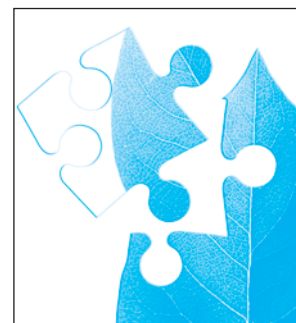
2. For each of the following tasks, write a set of numbered, step-by-step instructions (a solution) so complete that another person could perform the task without asking questions. Define the knowledge base of this person by listing what you expect the person to know in order to follow your directions. For example, for task “a” (below), make a cup of cocoa, the knowledge base might include such things as knowledge of milk or water, a refrigerator, pan, spoon, cocoa, cup, range top or microwave, and so forth.
 - a. Make a cup of cocoa.
 - b. Sharpen a pencil.
 - c. Walk from the classroom to the student lounge, your dorm, or the cafeteria.
 - d. Start a car (include directions regarding what to do if the car doesn’t start).
 - e. Get a glass of water from your kitchen.
 - f. Start your computer.
3. Test your solution in problem 2 by giving your instructions to another person to see whether he or she can accomplish the task without your help. If they can’t, modify your solution so that the person can accomplish the task. Check the solution again by giving the instructions to another person.
4. Develop the knowledge base for the following problems:
 - a. Shooting a gun
 - b. Building a house

- c. Going on a trek
 - d. Repairing a car
 - e. Calculating the cost of using your car for a month
 - f. Playing a video game.
5. Write a solution to the problem of finding the smallest number out of three numbers. List the specific steps that would enable another person to find the largest among three numbers presented.
 6. Which of the solutions in Problem 2 are harder to develop than others? Why do you think they are harder?
 7. Appendix A contains problems dealing with Otto the Robot. Otto has a limited number of tasks that he can accomplish. Discuss the approach you would take to develop a set of instructions that will allow Otto to accomplish the problems presented in Appendix A.
 8. Develop solutions for Otto's problems presented in Appendix A.
 9. How does your problem solving differ between finding a solution to a problem for your own life and that of designing a solution for one of Otto's problems?

This page intentionally left blank

Chapter 2

Beginning Problem-Solving Concepts for the Computer



Overview

Constants and Variables

Rules for Naming and Using Variables

Data Types

Numeric Data

Character Data—Alphanumeric Data

Logical Data

Other Data Types

Rules for Data Types

Examples of Data Types

How the Computer Stores Data

Functions

Operators

Expressions and Equations

Examples

Objectives

When you have finished this chapter, you should be able to:

1. Differentiate between variables and constants.
2. Differentiate between character, numeric, and logical data types.
3. Identify operators, operands, and resultants.
4. Identify and use functions.
5. Identify and use operators according to their placement in the hierarchy chart.
6. Set up and evaluate expressions and equations using variables, constants, operators, and the hierarchy of operations.

Although problems that arise in daily life are of many types, problems that can be solved on computers generally consist of only three: (1) computational, problems involving some kind of mathematical processing; (2) logical, problems involving relational or logical processing, the kinds of processing used in decision making on the computer; and (3) repetitive, problems involving repeating a set of mathematical and/or logical instructions. This chapter explains some computer fundamentals and demonstrates ways to set up expressions and equations to solve these types of problems on the computer. Programmers, need to know these computer fundamentals (see Figure 2.1).

Two of the most fundamental concepts that you will learn in this chapter are the constant and the variable. A programmer takes the data, the unorganized facts, and the information, the organized facts, relevant to a problem and defines them as constants or variables. They are the building blocks of the equations and expressions that ultimately make up solutions to computer problems. The programmer defines each constant and variable in a problem solution as a particular data type, such as numeric or character.

Other concepts that are essential to developing computer solutions to problems are operators and functions. Operators are the many signs and symbols that show relationships between constants and variables in the expressions and equations that make up the solution. The programmer has to know all of the many operators and how to use them. The order in which operators are processed is determined by a hierarchy that programmers need to know as well.

Operators are combined with constants and variables to create expressions and equations. Expressions and equations are used in instructions that are the building blocks of the solution. Functions are sets of instructions that are so commonly used that they are built into a computer language, saving the programmer the trouble of writing them.

These are key concepts. Without an understanding of how the computer uses and defines data, without knowing what the operators are, and without knowing how to use these concepts to construct expressions and equations, a programmer cannot effectively use the computer to solve problems.

When you study this chapter, it may help to keep these pointers in mind:

- Take each topic as it is presented, and learn the concepts pertaining to it.
- Understand the examples.
- Complete the questions and problems at the end of the chapter.
- Don't skip sections.
- Don't feel that something is too hard before you make an effort to understand it.
- Take each problem one step at a time.
- Don't skip steps.
- Don't assume anything.
- Don't be afraid to read a passage over again. Once may not be enough!

Concepts to Learn

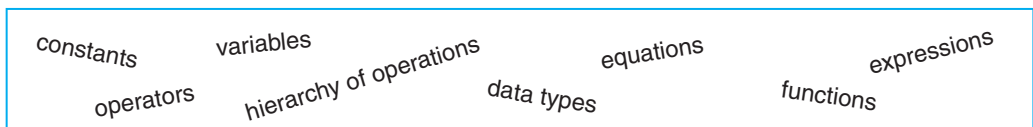


Figure 2.1 Important Concepts to Learn

Constants and Variables

constant

The computer uses constants and variables to solve problems. They are data used in processing. A **constant** is a value—that is, a specific alphabetical and/or numeric value—that never changes during the processing of all the instructions in a solution. Constants can be any type of data—numeric, alphabetical, or special symbols. (Data types are explained in more detail later.) In some programming languages and applications, constants can be named. In this case, the constant is given a location in memory and a name. During the execution of the program, this constant is given a value and then is referred to by its name. Once the constant is given a value, it cannot be changed during the execution of the program. For example, because the value of PI does not change, it would be a constant and defined within the program. This constant may be given a name, but the only way to change the value of the constant is to change the program. Any constant may be given a name. This allows an easier access to constants. Many name conventions stipulate that named constants be given names containing all capital letters in order to easily distinguish them from variables. However, this may vary between companies in which you may work.

variable identifier

In contrast, the value of a **variable** may change during processing. In many languages variables are called **identifiers** since the name identifies what the value represents. You need to be aware of this when learning the syntax and terminology of each language. In this text the word variable will be used. A programmer must give a name to each variable value used in a solution. The programmer uses a variable name as a reference name for a specific value of the variable. In turn, the computer uses the name as a reference to help it find that value in its memory. The computer sets up a specific memory location to hold the value of each variable name found in a program. Variables can be any data type, just as constants can. For instance, consider the cost of a pair of shoes. This data item should be given a variable name because the cost of a pair of shoes may change during the processing of the program or during multiple executions of the program. The variable name should be consistent with what the value of the variable represents. In this case, the name of the variable is *ShoeCost* because it represents the cost of the pair of shoes:

ShoeCost

56.00

However, if the cost of the shoes changes during the next execution of the program, then the value of *ShoeCost* will change but the variable name will not:

ShoeCost

35.00

Notice in Table 2.1 that when the value of a constant or a variable contains alphanumeric data—that is, numbers and/or special symbols—it is surrounded by quotation marks. These marks indicate to the computer that the value is a datum, not a variable name. The computer must have some way to distinguish between the two. Also notice that the variable names stand for specific data. The variable name *Cash* has a sum of money as its value. The variable name *City* has a city name as its value. Finally, notice that there are no blank spaces between words in variable names.

Table 2.1 Constants and Variables on the Computer

Constants	Variables
Rules: Constants cannot be changed. Examples: Value 25 Value −1.5 Value “ARCATA” Value “95521” Named Constants Rules: A constant cannot be changed after it is initially given a value. Storage location given a name. Referred to by the given name. Example: PI 3.142857	Rules: Storage locations are given names. Values of the contents for name variable locations can be changed. Referred to by variable name in the instructions. Examples: Variable Name—<i>Age</i> Value 25 Variable Name—<i>Cash</i> Value 83.59 Variable Name—<i>City</i> Value “Eureka” Variable Name—<i>ZipCode</i> Value “95501”

Naming Conventions for Constants and Variables

Each company you work for will have naming conventions for variables and module names. The convention for naming variables may differ with companies as well as languages. It is very important to have all programmers within an environment follow the specified conventions, and it may be a requirement for employment.

There are many reasons to follow a naming convention. First, it allows several programmers to work on the same project without the problem of conflicting variable and module names. Second, it allows programs to be easily read because there is only one consistent name for a variable. It also increases the readability between applications because the form of the variable name is consistent within a company. Third, naming conventions allow the code to be easily maintained. Programmers spend most of their time on software maintenance, not development; therefore, having a convention for naming variables decreases the time and increases the reliability when updating software. In addition, most software is maintained by many different people. Naming conventions allow the software to be transferred with the least difficulty. Fourth, the software should perform more efficiently by using consistent naming of modules and variables. Fifth, there should be an increase in performance expectation. And last, naming conventions should produce a clean, well-written program.

In a solution that calculates payroll for a company, the name of the company would be a constant since it does not change. The employee name, the hours, and the rate of pay would be variables because the values of these items change for each employee. If there were no such thing as a variable, the programmer would have to write a separate set of instructions for each employee. It is far more efficient to have one rather than a thousand programs to process payroll for 1,000 employees.

The rules for naming variables differ from language to language. However, the general rule is to name the variable as near to its meaning as possible: *PayRate* for the rate of pay, *Hours* for the number of hours worked during the pay period, and the like. Some languages and applications have character-length limitations or other restrictions for names (called reserved words), so adjustments have to be made as necessary. In this book, there will be no character-length limitations for variable names.

It is important to understand the difference between the name of a variable and the value of a variable. The *name* is the label the computer uses to find the correct memory location; the *value* is the contents of the location. The computer uses the variable name to find the location; it uses the value found at the memory location to do the processing.

It is also important to be consistent in the use of variable names because the computer will go only to the location with the specified name, regardless of whether it is the one intended by the user. For example, if you use *Hours* for hours worked, then you must use *Hours* at all times in referring to hours worked, not *Hrs* or *HoursWorked*. If the computer cannot find a memory location by the specified name, it will either name a new memory location and yield an incorrect result, or it will return an error message indicating there is no location as referenced and then cease execution.

Rules for Naming Constants and Variables

Rules for Naming and Using Variables

There are a number of rules for naming and using variables, as listed below. Table 2.2 illustrates some examples of incorrect variable names.

1. Name a variable according to what it represents, that is, *Hours* for hours worked, *PayRate* for rate of pay, and so on. Create as short a name as possible but one that clearly represents the variable.
2. Do not use spaces in a variable name; for example, use *HoursWorked*.
3. Start a variable name with a letter.
4. Do not use a dash (or any other symbol that is used as a mathematical operator) in a variable name. The computer will recognize these symbols as mathematical

Table 2.2 Incorrect Variable Names

Data Item	Incorrect Variable Name	Problem	Corrected Variable Name
Hours worked	<i>Hours Worked</i>	Space between words	<i>HoursWorked</i>
Name of client	<i>CN</i>	Does not define data item	<i>ClientName</i>
Rate of pay	<i>Pay-Rate</i>	Uses a mathematical operator	<i>PayRate</i>
Quantity per customer	<i>Quantity/customer</i>	Uses a mathematical operator	<i>QuantityPerCustomer</i>
6% sales tax	<i>6%_sales_tax</i>	Starts with a number	<i>SixPercentSalesTax</i> or <i>SalesTax</i>
Client address	<i>Client_address_for_client_of_XYZ_corporation_in_California</i>	Too long	<i>ClientAddress</i>
Variable name Introduced as <i>Hours</i>	<i>Hrs</i>	Inconsistent name	<i>Hours</i>
Variable name Introduced as <i>Hours</i>	<i>Hours_worked</i>	Inconsistent name	<i>Hours</i>

operators, turn your variable into two or more variables, and treat your variable as a mathematical expression.

5. After you have introduced a variable name that represents a specific data item, this exact variable name must be used in all places where the data item is used. For example, if the data item *hours worked* has the variable name of *Hours*, *Hours* must be used consistently. You may not use *Hrs* or *HoursWorked* to represent the same data item. If you do, the computer views these variables as new and different data items and will assign a new memory location to the new name.
6. Be consistent when using upper- and lower-case characters. In some languages *HOURS* is a different variable name than *Hours*.
7. Use the naming convention specified by the company where you work. As stated, in this text our naming convention is upper case for the first character in each of the words in the name, with no spaces between words in the name. Named constants will be in all upper case characters. For example:

Variable Names

PayRate
Rate
HoursWorked
Amount
Temperature

Constant Names

PI
KCONSTANT
MULTIPLIER

Your instructor may want you to use a different naming convention. Be sure you follow the convention exactly.

Data Types

data

information

To process solutions, the computer must have data. **Data** are unorganized facts. They go into the computer as input and are processed by the program. What is returned to the user is output, or **information**. This information is printed in the form of reports. For example, when a computer calculates the balance of a checkbook, data are the checks, the deposits, and the bank charges. The information from the processing is shown on the balance sheet (see Figure 2.2).

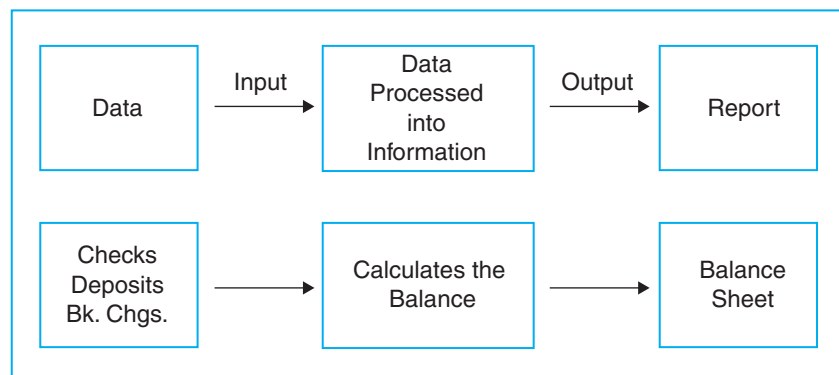


Figure 2.2 Processing Data—How a Computer Balances a Checkbook

The data the computer uses are of many different types. Computers must be told the **data type** of each variable or constant. The most common data types are numeric, character, and logical. A few languages and applications also use the date as a data type. Other languages allow the programmer to define data types.

Numeric Data

Numeric data include all types of numbers (see Table 2.3). Numeric is the only data type that can be used in numeric calculations. The subtypes of numeric data include *integers* and *real numbers*. Integers are whole numbers, such as 5,297 or −376. They can be positive or negative. Real numbers, or floating point numbers, are whole numbers plus decimal parts. In problem solving on the computer, programmers use integers when there is no reason for using partial numbers, as when they are designing counters, expressions used for counting things, such as inventory items or people. A real number can be expressed in scientific notation, such as 2.3E5 or 5.4E−3. In scientific notation, the *E* stands for *times 10 to the power of*. Therefore, 2.3E5 is the same number as 2.3×10^5 , or 230,000.0, and 5.4E−3 is the same number as 5.4×10^{-3} , or 0.0054. A number expressed in scientific notation is always considered a real data type. The computer does not use commas in a number during the processing of a calculation, only when the number is formatted as output for the user.

Numeric data are used for values, such as rate of pay, salary, angles, distance, or radius, that have calculations performed on them. Numbers such as an account number or a zip code, which would not have calculations performed on them, would not be designated by the programmer as numeric data.

Each data type has a *data set*, the set of symbols necessary to specify a datum as a particular data type. A data set is the set of values that are allowed in a particular data type. The data set for the numeric data type includes all base 10 numbers, the plus sign (+) and the negative sign (−). The data set for the integers includes all whole numbers within the limitations of the computer or the language, and for the real numbers, all whole numbers with decimal parts within the limitations of the computer or the language, including zero as a whole number and zero as a decimal part.

Table 2.3 Data Types and Their Data Sets

Data Type	Data Set	Examples
Numeric: Integer	All whole numbers	3580 −46
Numeric: Real	All real numbers (whole + decimal)	−3792.91 4739416.0 0.00246
Character (surrounded by quotation marks)	All letters, numbers, and special symbols	“A” “a” “M” “z” “k” “1” “5” “7” “8” “0” “+” “−” “(” “o/” “\$”
String (surrounded by quotation marks)	Combinations of more than one character	“Arcata” “95521” “707-444-5555”
Logical	True False	True False

Character Data—Alphanumeric Data

character data

The **character data** set, sometimes called alphanumeric data set, consists of all single-digit numbers, letters, and special characters available to the computer—a, A, Z, 3, #, &, and so forth—placed within quotation marks. An upper case letter is considered a different character from a lower case letter. The ASCII (American Standard Code for Information Interchange) character set contains 256 characters. The first 128 comprise a standard set (see Appendix C for the ASCII and EBCDIC data codes), and the second 128 differ with each computer. Characters cannot be used for calculations even if they consist of only numbers. When more than one character are put together, the computer considers this item a *string*—derived from *a string of characters*. Some languages do not differentiate between characters and strings. All character data are considered string data.

String Data

Character and string data can be compared and arranged in alphabetical order in the following way. The computer gives each character a number. This numeric representation is necessary because the computer works only with numbers. The numbers are compared to see which is larger and are then arranged in ascending numeric order. Since *B* has a larger number representing it than *A*, *B* is placed after *A*. *C* would follow *B* by the same method, and so forth. String data can be tested in the same way to alphabetize names, cities, and the like. *Banana* is larger than *Apple* because *B* has a larger number representing it than *A*. *Joan* is larger than *James* since the letter *o* is attributed a larger number than *a*. Upper case letters have smaller numeric representations than lower case letters. It is important to bear this difference in mind when you are comparing letters to see if they are equal. Languages that differentiate between lower case and upper case are called case sensitive. When they are not case sensitive, upper- and lower case letters are treated the same. You can use functions before the comparison to ensure that data are not case sensitive by making all letters either upper- or lower case.

concatenation

Character data or string data can be joined together with the + operator in an operation called **concatenation**. When two pieces of character data are joined, the concatenation results in the second being placed at the end of the first, as in “4” + “4” = “44” (not “8”). Concatenation could be used to join a first name with a last name or to join pieces of data together to make a code for an inventory item. The concatenation operator varies with each language.

Most data in business—names, account numbers, addresses, cities, states, telephone numbers, zip codes—are string data. As a rule, items that would not have mathematical computations performed on them should be designated string data types. There is another reason for designating a zip code as string data. If you are using numeric data, there is no way to hold onto leading zeros, that is, the zeros at the front of a number, such as 00987. As a result, a zip code on the East Coast could not be entered correctly as a numeric datum. The leading zeros could be preserved only with string data.

Logical Data

logical data

Logical data consist of two values in the data set—the words *True* and *False*. (Some languages accept *yes*, *T*, and *Y* for *True*, and *no*, *F*, and *N* for *False* as part of the data set.) These are used in making yes-or-no decisions. For example, logical data type might be used to check someone’s credit record; *True* would mean her credit is okay, and *False* would mean it’s not okay. A home accounting system could use logical data type for check returns; *True* would mean the check has been returned, and *False* would mean it has not. There are many such uses for logical data. They are discussed further in later chapters. Remember, the logical data set *True* and *False* are

not string data and are considered reserved words—that is, word that cannot be used as variable names.

Other Data Types

Date Data Type

There are other data types available to most programmers, such as the date data type and user-defined data types. The date data type is a number for the date that is the number of days from a certain date in the past such as the first day of the 20th century or January 1, 1940. The use of this data type for the date allows the user to subtract one date from another date to calculate the number of days between dates. This simplifies the calculation when the dates cross over years or months. The date is printed in the date format instead of a number. The date data type is a numeric data type because you can perform mathematical calculations on any date.

User Defined Data Types

Programmers may define their own data types, such as data types that have as their data set soda pop brands, or types of cars, or computer components. The user must specify the items in the data set for each user-defined data type. If the data item is not contained within the data set, it is not part of the set. For example, if the programmer does not include *Mouse* as part of the data set for a user-defined data type of computer parts, then the computer does not recognize a *Mouse* as a computer part.

This book will use the date data type in a few places; however, user-defined data types will not be further discussed or used.

Rules for Data Types

Rules for Data Types

1. The data that define the value of a variable or a constant will most commonly be one of three data types: numeric, character (including character string), or logical.
2. The programmer designates the data type during the programming process. The computer then associates the variable name with the designated data type.
3. Data types cannot be mixed. For example, string data cannot be placed in a variable memory location that has been designated as numeric, and vice versa. When the computer expects a certain data type, the user must use that type or the computer will return an error message.
4. Each of the data types uses what is called a *data set* (see Table 2.3). The numeric data uses the set of all base 10 numbers, the plus sign (+), and the negative sign (−); the character type uses the set of all characters available to the computer; the logical data type uses the set of data consisting of the words *True* and *False*. The use of any data outside the data set results in an error.
5. Any numeric item that must be used in calculations resulting in a numeric result must be designated as numeric data type. All other numbers should be designated as character or character-string data types, even if data are all numbers, as in zip codes.

Examples of Data Types

Table 2.4 illustrates some common uses for numeric, character (including character string), and logical data types. Data are drawn from everyday life and business, and

Table 2.4 Examples of Data Types

Data	Data Type	Explanation
The price of an item: 7.39, 12.98	Numeric: real	The price of an item would be used in calculations. The price is money and needs decimals.
An account number: "A2453," "2987"	Character string	An account number consists of alphanumeric or simply numeric data. It is not used for calculations.
A quantity: 12389	Numeric: integer	A quantity is used for calculations. It is an integer because it normally is a whole number.
The name of a company: "Smith Corp."	Character string	A name of something is alphabetical and therefore would be character string data type.
A credit check: <i>True, False</i>	Logical	The credit check of a customer would be a choice of two answers such as <i>yes</i> , it is okay, or <i>no</i> , it is not okay. Therefore, it should be logical data.
A zip code: "95521" "76548" "00538"	Character string	A zip code would be a character string because there are no calculations to be done on a zip code, and the leading zeros need to be retained.
A date: 01/23/87 or "03/14/87"	Date or character string	If the date data type is available, then it is used; if not, then it has to be a string because it is alphanumeric.
A date: 187259	Numeric: integer	A calendar date is the number of days from a given date, such as the first day of the 20th century. It can be used for calculations.
Social Security Number "333-33-3333"	Character string	A SSN is string data because it is alphanumeric and is not used for calculations.

are examples of the kinds of data that are commonly used in solving various types of problems on the computer. To the right of the data examples are data types and explanations of why each type is appropriate to the data.

How the Computer Stores Data

Data Storage

The computer stores data internally in memory locations. These data are found by the variable names used by a program. Each variable name is given a memory location, and each memory location can hold one and only one value at a time. When a user enters a new value into the variable location, the previous value is destroyed. These memory locations are temporary, as the internal memory is a volatile memory. When a program completes its instructions, and/or when the computer is turned off, the values stored in the internal memory are destroyed.

Data and instructions are temporarily stored in the computer's internal memory during the processing. When data, information, or programs have to be kept for future

use, they are stored externally on an external storage medium such as a hard disk drive in storage areas called **files**. There are basically two types of files: program files and data files. Program files contain the instructions to tell the computer what to do. This book helps the programming student create programs that will be saved on the external storage device as program files. Data files contain the data required to execute the program files.

Functions

Functions are small sets of instructions that perform specific tasks and return values. They are usually built into a computer language. Functions are used as parts of instructions in a solution. Because they are basic tasks that are used repeatedly in the problem-solving process, by using them a programmer or user can shorten the problem-solving time and improve the readability of the solution. Each language has a set of functions within it. This set varies with the language and the computer. Most languages allow programmers to write their own functions. Libraries of functions can be added to many languages.

The form of a function is the name of the function followed by an open parenthesis, followed by the data needed to perform the function and concluded by a closed parenthesis:

FunctionName(data)

The value of the result of the function is returned in the name of the function.

Functions use data. The data is listed as part of the function and are called **parameters**. Functions normally do not alter the parameters. Take the square root function, *Sqrt(N)*. This function will calculate the square root of *N*. *Sqrt* is the name of the function. *N* is the data needed to calculate the square root and, therefore, it is the parameter. Parameters are surrounded by parentheses. The maximum function, *Max(N1, N2, N3)*, will find which of three numbers is the largest. The name of the function is *Max*. The parameters, data surrounded by parentheses and needed to do the calculation, are *N1*, *N2*, and *N3*. Not all functions need parameters. The function *Random* generates or calculates a random number. No data are needed to do this calculation, so there are no parameters. A parameter can be a constant, a variable, or an expression. (An expression is a calculation, such as *interest/100*, which has not been given a permanent memory location in the computer.) The names of the functions may vary from language to language. This book will give a generic name to each function and list the parameters in parentheses.

Table 2.5 lists and defines a few of the basic functions that are found in many languages. Functions are unique to each language. As a programmer, you need to investigate the functions used in the language in which you are writing your solution. These functions have been divided into classes.

Types of Functions

1. *Mathematical functions.* Often used in science and business, mathematical functions calculate such things as square root, absolute value, or a random number. Other mathematical functions used primarily for scientific purposes have not been included in the table.
2. *String functions.* These are used to manipulate string variables. For example, they copy part of the string into another variable, find the length or the number of characters in the string, and so forth.

Table 2.5 Functions (*continued on page 35*)

Function*	Definition	Example	Result
Mathematical Functions			
<i>Sqrt(N)</i>	Returns the square root of <i>N</i> .	<i>Sqrt(4)</i>	2
<i>Abs(N)</i>	Returns the absolute value of <i>N</i> .	<i>Abs(-3)</i>	3
<i>Round(N, n1)</i>	Returns the rounded value of <i>N</i> to the <i>n1</i> place.	<i>Round(3.7259,2)</i>	3.73
<i>Integer(N)</i>	Returns the closest whole number less than or equal to <i>N</i> .	<i>Integer(5.7269)</i>	5
<i>Random</i>	Returns a random number between 0 and 1, but not 1. This number is mathematically generated. To find a number <i>N</i> between <i>N1</i> and <i>N2</i> , inclusive, use the following formula: $N = \text{Integer}(\text{Random} * (N2 - N1 + 1)) + N1$, where <i>N1</i> is the smallest number and <i>N2</i> is the largest number.	<i>Random</i>	0.239768
<i>Sign(N)</i>	Returns the sign of <i>N</i> : 1 when <i>N</i> is positive, 0 when <i>N</i> is zero, -1 when <i>N</i> is negative.	<i>Sign(7.39)</i>	1
String Functions			
<i>Mid(S, n1, n2)</i>	Returns a set of <i>n2</i> characters starting at <i>n1</i> in the string <i>S</i> .	<i>Mid(S, 3, 2)</i> where <i>S</i> = "Thomas"	"om"
<i>Left(S, n)</i>	Returns a set of <i>n</i> characters on the left side of the string <i>S</i> .	<i>Left(S, 3)</i> where <i>S</i> = "Thomas"	"Tho"
<i>Right(S, n)</i>	Returns a set of <i>n</i> characters on the right side of the string <i>S</i> .	<i>Right(S, 3)</i> where <i>S</i> = "Thomas"	"mas"
<i>Length(S)</i>	Returns the number of characters in the string <i>S</i> .	<i>Length(S)</i> where <i>S</i> = "Thomas"	6

*Definitions of symbols:

N is a numeric value—a constant, a variable, or an expression

S is a string value—a constant, a variable, or an expression

n, *n1*, *n2* are integer values—a constant, a variable, or an expression

3. *Conversion functions.* These functions are used to convert data from one data type to another. For example, since character strings cannot be used in calculations, one of these functions would convert a string value to a numeric value.
4. *Statistical functions.* These functions are used to calculate things such as maximum values, minimum values, and so forth.
5. *Utility functions.* This class is very important in business programming because most reports require some use of utility functions. They access information outside the program and the language in the computer system. Examples of these include date and time functions.

Table 2.5 (Continued from page 34)

Function*	Definition	Example	Result
Conversion Functions (change data type)			
<i>Value(S)</i>	Changes a string value into a numeric value.	<i>Value</i> ("57.39")	+57.39
<i>String(N)</i>	Changes a numeric value into a string value.	<i>String</i> (+57.39)	"57.39"
Statistical Functions			
<i>Average(list)</i>	Returns the average of a list of numbers.	<i>Average</i> (5, 3, 8, 6)	5.5
<i>Max(list)</i>	Returns the maximum value from a list of numbers.	<i>Max</i> (5, 3, 8, 6)	8
<i>Min(list)</i>	Returns the minimum value from a list of numbers.	<i>Min</i> (5, 3, 8, 6)	3
<i>Sum(list)</i>	Returns the sum of a list of numbers.	<i>Sum</i> (5, 3, 8, 6)	22
Utility Functions			
<i>Date</i>	Returns the current date from the system. The date may be in various forms: <i>mm/dd/yy</i> , day only, month only, year only, or Julian calendar.	<i>Date</i>	09/15/98
<i>Time</i>	Returns the current time from the system. The time may be in various forms: <i>hh:mm:ss</i> , seconds from midnight, or minutes from midnight.	<i>Time</i>	9:22:38
<i>Error</i>	Returns control to the program when a systems error occurs.		

Note: Other functions are listed in Appendix F.

Table 2.5 is only a partial list of functions. There are many other functions that are not listed here and may not be universal. Many languages have hundreds of functions for the programmer's use. As a programmer, you should check the function list as to the availability of the types of functions you might want to use. You also need to note that function names are not allowed to be used as variable, constant, or module names. They are considered reserved words. Using these reserved words will cause a bug or error in a program.

Operators

operator

The computer has to be told how to process data. This task is accomplished through the use of operators. **Operators** are the data connectors within expressions and equations. They tell the computer how to process the data. They also tell the computer what type of processing (mathematical, logical, or whatever) needs to be done. The types of operators used in calculations and problem solving include mathematical, relational, and logical operators. Without these operators very little processing can be done.

Operand
Resultant

The *operand* and the *resultant* are two concepts related to the operator. Operands are the data that the operator connects and processes. The resultant is the answer that results when the operation is completed. For example, in the expression $5 + 7$, the $+$ is the operator, 5 and 7 are the operands, and 12 is the resultant. Operands can be constants or variables. The data type of the operands and the resultant depends on the operator.

Mathematical operators include addition, subtraction, multiplication, division, integer division, modulo division, powers, and functions. The computer has a symbol for each of them (see Table 2.6). You are probably familiar with addition, subtraction, multiplication, division, and powers. However, you may not be familiar with integer and modulo division.

These two operations are related. In integer division, the resultant is the whole number in the quotient. In modulo division, the resultant is the whole number remainder. These two types of division are used in business to find the hours and minutes someone has worked given the total number of minutes, or the days and weeks worked given the total number of days. For example, if Jane Smith has worked 19 days during the month, then, assuming a 5-day work week, she has worked $19 \setminus 5 = 3$ weeks and $19 \text{ MOD } 5 = 4$ days. Therefore, she has worked 3 weeks and 4 days. When dividing 19 by 5, the result is 3 with a remainder of 4. The 3 is the resultant of the integer division and the 4 is the resultant of the modulo division. If the dividend is less than the divisor, such as $28 \text{ MOD } 379$, then the resultant is always equal to the dividend, in this case 28. The resultant of $28 \setminus 379$ is zero because the dividend is less than the divisor. When you divide 28 by 379, your resultant is zero with a remainder of 28.

Table 2.6 Operators and Their Computer Symbols

Operator	Computer Symbol	Example	
Mathematical		Operation	Resultant
Addition	+	$3.0 + 5.2$	8.2
Subtraction	−	$7.5 - 4.0$	3.5
Multiplication	*	$8.0 * 5.0$	40.0
Division	/	$9.0/4.0$	2.25
Integer division	\	$9 \setminus 4$	2
Modulo division	MOD	$9 \text{ MOD } 4$	1
Power	^	$3 ^ 2$	9
Relational			
Equal to	=	$5 = 7$	<i>False</i>
Less than	<	$5 < 7$	<i>True</i>
Greater than	>	$5 > 7$	<i>False</i>
Less than or equal to	<= (two key strokes)	$5 <= 7$	<i>True</i>
Greater than or equal to	>= (two key strokes)	$5 >= 7$	<i>False</i>
Not equal to	<> (two key strokes)	$5 <> 7$	<i>True</i>
Logical			
Not	NOT	NOT <i>True</i>	<i>False</i>
And	AND	<i>True</i> AND <i>True</i>	<i>True</i>
Or	OR	<i>True</i> OR <i>False</i>	<i>True</i>

Relational operators include the following: equal to, less than, greater than, less than or equal to, greater than or equal to, and not equal to. A programmer uses relational operators to program decisions. The operands of a relational operator can be either numeric or character (a string); however, both operands must be of the same data type. The resultant of a relational operation is logical data type *True* or *False*. The programmer designs one action or set of actions that will follow when a relational expression is *True*, and another action or set of actions that will follow when the expression is *False*. The use of relational operators is the only way for the computer to make decisions.

For example, when a credit card customer’s balance is *less than \$500 (True)*, then the customer can charge another purchase. When the balance is *not less than \$500 (False)*, then he cannot charge another purchase. The expression would be set up as *Balance < 500*. The operands are *Balance* and *500*; the operator is *<*; the resultant is either *True* or *False* depending on the value of the balance. The programmer would use this expression in what is called a *decision instruction*.

Relational operators are also used to control repetitive instructions called *loops*. A set of instructions to enter data for a client, which would be repeated until the data are entered for every client, is one example of a loop. When the computer processes this type of instruction, the loop repeats until the resultant changes from *True* to *False*, or vice versa. In the example, the resultant would change when the computer can find no more client data to enter.

Logical operators are the third type of operator (see Table 2.7). Logical operators are used to connect relational expressions (decision-making expressions) and to perform

Table 2.7 Definitions of the Logical Operators

NOT							
A	Not A	When A Is		The Resultant Is			
True	False	NOT True		Is		False	
False	True	NOT False		Is		True	

AND							
A	B	A AND B	When A Is		When B Is		The Resultant Is
True	True	True	True	AND	True	Is	True
True	False	False	True	AND	False	Is	False
False	True	False	False	AND	True	Is	False
False	False	False	False	AND	False	Is	False

OR							
A	B	A OR B	When A Is		When B Is		The Resultant Is
True	True	True	True	OR	True	Is	True
True	False	True	True	OR	False	Is	True
False	True	True	False	OR	True	Is	True
False	False	False	False	OR	False	Is	False

OR Operator

operations on logical data. For example, a store might require a driver’s license or a check-cashing card on file for a customer to cash a check. When the customer has a driver’s license, the check can be cashed. When she has a check-cashing card, the check can be cashed. The expression is written as *License OR Card*. *License* and *Card* are the operands. They are logical data; that is, the value of each is *True* or *False*. The operator is OR. The resultant is *True* or *False* depending on the values of the operands. When one or both of the operands is *True*, then the resultant is *True*. When both of the operands are *False*, then the resultant is *False*.

AND Operator

The OR operator is one of three logical operators. The others are AND and NOT. When the AND operator is used, the resultant is *True* only when both of the operands are *True*. When either or both of the operands are *False*, the resultant is *False*. A programmer uses AND when two requirements must be *True* in order for an action or set of actions to take place, such as when a store requires a customer to have *both* a driver’s license *and* a check-cashing card to cash a check. The NOT operator is the only logical operator that requires only one operand. The resultant of the NOT operator changes in an operand from *True* to *False*, or from *False* to *True*. A programmer uses the NOT operator to change an operand to the opposite value. This operation is sometimes called *reversing* the value of the operand.

NOT Operator

hierarchy

These mathematical, relational, and logical operators have a **hierarchy**, or precedence, an order in which their operations take place (see Table 2.8). To reorder the normal

Hierarchy of Operations

Table 2.8 Hierarchy of Operations

Order of Operations	Operand Data Type	Resultant Data Type
() Reorders the hierarchy; all operations are completed within the parentheses using the same hierarchy.		
1. Functions		
Mathematical Operators		
2. Power	Numeric	Numeric
3. \, MOD	Numeric	Numeric
4. *, /	Numeric	Numeric
5. +, -	Numeric	Numeric
Relational Operators		
6. =, <, >, <=, >=, <>	Numeric or string or character	Logical
Logical Operators		
7. NOT	Logical	Logical
8. AND	Logical	Logical
9. OR	Logical	Logical

processing sequence, the programmer uses parentheses. The processing of the operands (as directed by the operators) always starts with the innermost parentheses and works outward, and processes from left to right. Each level of operators within a set of parentheses requires that the computer make another pass through the parentheses, until all levels have been cleared. This hierarchy is important to the programmer because the order of the operations determines the result of the expression. If the operations are not completed in the correct order, the result of the expression may be incorrect. For example, assuming a 40-hour work week and overtime pay at 1.5 times regular pay, the expression to calculate overtime pay would subtract 40 from the hours worked and multiply the result by the regular wage times 1.5. The expression would be written as

$$(Hours - 40) * Wage * 1.5$$

As you can see from the hierarchy chart, multiplication is processed before subtraction, so parentheses must be added to tell the computer to do the subtraction first, before the multiplication. If the parentheses are not added, the result will be incorrect.

In this hierarchy table the integer and modulo divisions are on a separate level from real number multiplication and division. However, many languages place these four operators on the same level. When this is true, it is important to place parentheses around the integer or modulo operations so that the integer and modulo divisions are executed before the multiplication and division. In the following example you will get two different results depending upon whether you execute the integer and modulo division on the same level or separate them with parentheses to force different levels. Look at the equation:

$$F = 6 * 2 \setminus (6 + 2)$$

If the multiplication and integer division is on the same level, the answer will be 1. When they are on different levels, where the integer division will be executed first, the answer will be 0. Remember that the execution of operators on the same level is completed from left to right. To force the integer division to be executed first, resulting in a zero resultant, the equation would have to be changed to:

$$F = 6 * (2 \setminus (6 + 2))$$

Check with the language you are working with to find out whether integer and modulo divisions are on the same or different levels. In this book we will assume they are on the different levels. However, please be aware of the difference when starting to develop equations and expressions for the language you are learning.

Each type of operator requires a certain data type for operands and determines the data type of the resultant. Mathematical operators require numeric data as the operands and have numeric resultants. When any of the relational or logical operators are used (regardless of whether they are used in conjunction with mathematical operators), the resultant is logical data. Relational operators use numeric, character, or string data types as the operands; however, the data type of the operands of an operator must be the same. The logical operators require logical data for operands.

Expressions and Equations

A knowledge of constants and variables, of the three data types, and of operators, operands, and resultants is not very valuable until you can use these concepts to create expressions and equations. The problem you are trying to solve may be calculating pay,

including tax, Social Security, and medical deductions. It may be an inventory problem with reorder quantities and cash values. It may be a problem of calculating interest or payments on a loan, or of putting a mailing list in order. These tasks all require the use of different operators and would be written as expressions and equations in order for the computer to process the data and arrive at a result. There are very few computer problems that do not use expressions and equations in their solutions.

expression

Expressions and equations make up part of the instructions in the solution to a computer problem. An **expression** processes data, the operands, through the use of operators. For example, to find the number of square feet in a room you would multiply the length of the room by the width in the expression

$$\text{Length} * \text{Width}$$

equation

An **equation** stores the resultant of an expression in a memory location in the computer through the equal (=) sign. The expression above would be used as part of an instruction in the equation

$$\text{Area} = \text{Length} * \text{Width}$$

The resultant of the expression $\text{Length} * \text{Width}$ would then be stored in a memory location called *Area*.

As you can see in Table 2.9, expressions use no equal sign (except as a relational operator) and are used as part of an instruction, such as part of an equation. The resultant of an expression is not stored and, therefore, is unavailable for use at another time. An expression can use numeric, string, character, or logical data types.

Equations are often called *assignment statements* because the variable on the left-hand side of the equal sign is assigned the value of the expression on the right-hand side. The equal sign does not mean *equals*; instead, it means *replaced by* or *is assigned the value of*. This distinction is important because it is possible to have equations on the computer that are not allowed in a mathematics class, such as $N = N + 1$ (one is added to the old value of N to get a new value of N). This equation is the way the computer counts. There is only one variable on the left-hand side of the equal sign. On the right-hand side of the equal sign is an expression. The right-hand side is processed before the assignment is made (see Table 2.9 and the examples that follow).

For the beginning programmer, a big part of learning to solve problems on the computer is learning how to write and evaluate expressions and equations. (Recall that

Table 2.9 Expressions and Equations

Expressions	Equations
$A + B$ <i>A</i> and <i>B</i> are numeric. The resultant is numeric and is not stored.	$C = A + B$ <i>C</i> , <i>A</i> , and <i>B</i> are numeric. The resultant is stored in <i>C</i> .
$A < B$ <i>A</i> and <i>B</i> are numeric, character, or string. The resultant is logical and is not stored.	$C = A < B$ <i>A</i> and <i>B</i> are numeric, character, or string. The resultant is stored in <i>C</i> ; <i>C</i> is logical.
$A \text{ OR } B$ <i>A</i> and <i>B</i> are logical. The resultant is logical and is not stored.	$C = A \text{ OR } B$ <i>C</i> , <i>A</i> , and <i>B</i> are logical. The resultant is stored in <i>C</i> .

to *evaluate* means to test for correctness using actual data.) The following examples illustrate how to use the concepts you've learned in this chapter to write and evaluate expressions and equations.

Example 1: Setting Up a Numeric Expression

A programmer will often be given an expression to use in solving a problem. Assume the programmer has to modify the following mathematical expression for computer use:

$$X(3Y + 4) - \frac{4Y}{X + 6}$$

The appropriate computer expression would be the following:

$$X * (3 * Y + 4) - 4 * Y / (X + 6)$$

All variables, constants, and operators have to be on the same line. There must be an operator between variables and/or constants. For the computer, there are no assumed multiplication signs. Parentheses are added when the hierarchy has to be reordered or when parentheses would improve readability.

Follow these steps to complete the expression:

1. Write all parentheses, operands, and operators on a single line with the dividend first, followed by the divisor:

$$X(3Y + 4) - 4Y/X + 6$$

2. Insert all implied operators. The computer does not use any implied operators as used in mathematics. These usually include multiplication and power signs.

$$X * (3 * Y + 4) - 4 * Y/X + 6$$

3. Insert all parentheses where the hierarchy needs to be reordered. $X + 6$ needs to be calculated before completing the division, therefore parentheses need to be inserted around the $X + 6$.

$$X * (3 * Y + 4) - 4 * Y / (X + 6)$$

Example 2: Setting Up a Mathematical Equation

A mathematical equation might be given to the programmer in the following form:

$$Y + 3 = X(Z + 5)$$

The programmer has to modify the equation so that it is in the form of an assignment instruction:

$$Y = X * (Z + 5) - 3$$

Notice that there is only one variable (and no calculations) on the left side and an expression on the right side of the equal sign.

Follow these steps to complete the equation:

1. Write all parentheses, operands, and operators on a single line:

$$Y + 3 = X(Z + 5)$$

2. Use mathematical rules to complete the equation so that there is one variable on the left side of the equal sign. In this case, subtract 3 from each side:

$$\begin{aligned}Y + 3 - 3 &= X(Z + 5) - 3 \\Y &= X(Z + 5) - 3\end{aligned}$$

3. Follow the steps in Example 1 to complete the equation.

$$Y = X * (Z + 5) - 3$$

Example 3: Setting Up a Relational Expression

A relational expression is used to make decisions. Given the expression

$$X \text{ is less than } Y + 5$$

the programmer would change its form to the following:

$$X < Y + 5$$

Because this is a relational expression, either a numeric variable, a numeric constant, or a mathematical expression is on either side of the operator. X and Y would be the same data type, in this case, a numeric data type. The resultant would be a logical data type—*True* or *False*, depending on the values of X and Y .

Example 4: Setting Up a Logical Expression

Often a computer decision is based on more than one *True* or *False* operand. For example, in order to cash a check a customer must have a driver's license (A) or a check-cashing card (B) on file at the store. As is the case with all logical expressions, this case would require a logical operator, $A \text{ OR } B$. The operands, A and B , are logical data type; it is either *True* or *False* that the customer has a driver's license, and *True* or *False* that she has a check-cashing card. The resultant is also a logical type: The customer can cash a check (*True* or *False*).

Example 5: Evaluating a Mathematical Expression

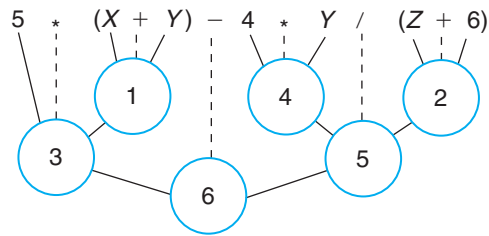
To find out if proposed solutions are correct, it is important for the programmer to evaluate, or test, all expressions and equations. Assume the programmer has written the expression

$$5 * (X + Y) - 4 * Y / (Z + 6)$$

The programmer uses the following values to evaluate the expression:

$$X = 2 \quad Y = 3 \quad Z = 6$$

Notice the structure of the evaluation that follows. A dashed line leading to a circle indicates the operator. The solid lines indicate the operands. Following the illustration of the equation is a list of the operations and the resultant of each operation. The number of circles should equal the number of operators. Be sure the circles using other circles are below the circles they need to use, that is, 3 is lower than 1, 6 is lower than 3 and 5.



Operation	Resultant
1. $X + Y$	5
2. $Z + 6$	12
3. $5 * \text{resultant of 1}$	25
4. $4 * Y$	12
5. Resultant of $4 / \text{resultant of 2}$	1
6. Resultant of $3 - \text{resultant of 5}$	24

Illustrating the structure of the expression and listing the operations and results in this way simplifies the evaluation process for the programmer. The evaluation must be completed according to the hierarchy of operations. The operations within the parentheses would be completed first, moving from left to right—so $X + Y$ would be completed first, and $Z + 6$ second. According to the hierarchy, the programmer would then complete the multiplications and the divisions from left to right. The division would be completed after the multiplications in this expression only because it is to the right. The final operation to be completed would be the subtraction.

This evaluation shows how to test a computer expression to see if it matches an equivalent mathematical expression. It gives the programmer a correct result that can be compared to the results manually calculated by the client, or by whoever requested the computer solution to the problem, so one can be sure that the solution is correct.

Example 6: Evaluating a Relational Expression

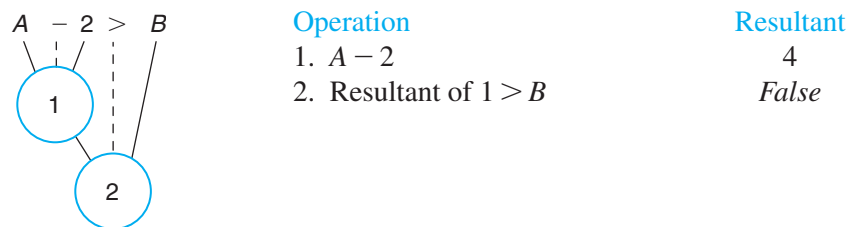
Assume the programmer has written the expression

$$A - 2 > B$$

The programmer uses the following values to evaluate the expression:

$$A = 6 \quad B = 8$$

The following illustration shows the structure of the evaluation:



According to the hierarchy of operations, the mathematical operation is completed first.

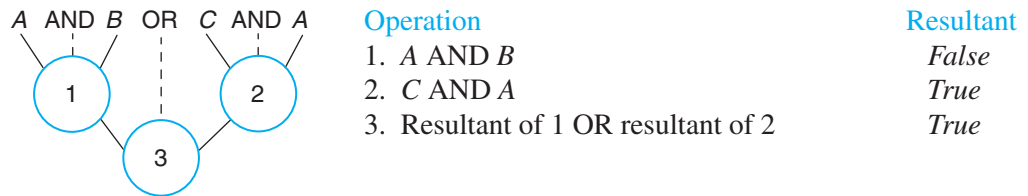
Example 7: Evaluating a Logical Expression

Assume the programmer has written the expression

$$A \text{ AND } B \text{ OR } C \text{ AND } A$$

The programmer uses the following values to evaluate the expression:

$$A = \text{True} \quad B = \text{False} \quad C = \text{True}$$



Example 8: Evaluating an Equation That Uses Both Relational and Logical Operators

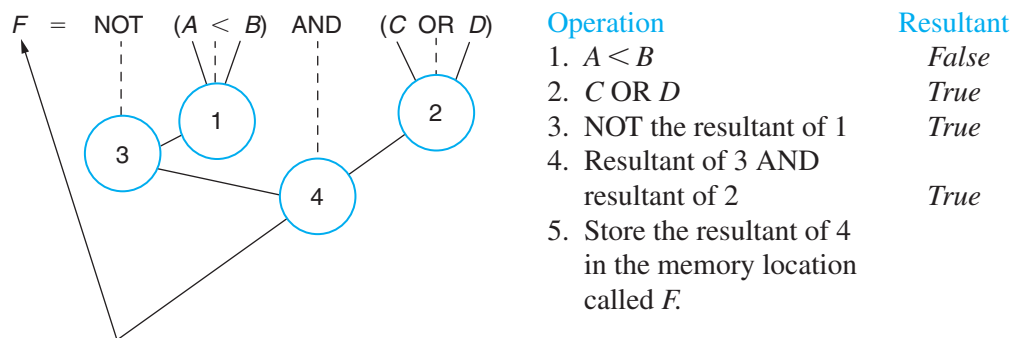
Assume the programmer has written the following equation:

$$F = \text{NOT}(A < B) \text{ AND } (C \text{ OR } D)$$

The programmer uses the following values to evaluate the equation:

$$A = 4 \quad B = 2 \quad C = \text{True} \quad D = \text{False}$$

The operations are completed in hierarchical order from left to right, as illustrated in the figure.



Example 9: Developing a Table of All Possible Resultants of a Logical Expression

It is often valuable for a programmer to figure out the resultants for all of the possible combinations of values of the variables in an expression. To create a table of all possible combinations, double the number of combinations from a table with one less unknown. For example:

- One unknown— A . Two combinations: A can be either *True* or *False*.

A
<i>True</i>
<i>False</i>

- Two unknowns— A and B . Four combinations: B can be either *True* or *False* for each value of A .

A	B
<i>True</i>	<i>True</i>
<i>True</i>	<i>False</i>
<i>False</i>	<i>False</i>
<i>False</i>	<i>True</i>

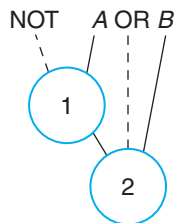
- Three unknowns— A , B , and C . Eight combinations.

A	B	C
<i>True</i>	<i>True</i>	<i>True</i>
<i>True</i>	<i>True</i>	<i>False</i>
<i>True</i>	<i>False</i>	<i>True</i>
<i>True</i>	<i>False</i>	<i>False</i>
<i>False</i>	<i>True</i>	<i>True</i>
<i>False</i>	<i>True</i>	<i>False</i>
<i>False</i>	<i>False</i>	<i>True</i>
<i>False</i>	<i>False</i>	<i>False</i>

Given the expression

NOT A OR B

the following table gives the resultants for all combinations of the values of A and B . The equation to find the number of combinations is 2^n , where n is the number of variables. For example, with two variables, A and B , there are four combinations; and with three variables, A , B , and C , there are eight combinations.



A	B	NOT A OR B
<i>True</i>	<i>True</i>	<i>True</i>
<i>True</i>	<i>False</i>	<i>False</i>
<i>False</i>	<i>True</i>	<i>True</i>
<i>False</i>	<i>False</i>	<i>True</i>

Example 10: Developing a Logical Expression from a Given Problem

Many times a programmer will be given the policies of a company and then be required to set up a logical expression from those policies. To do this, the programmer first needs to list the items on which the condition is dependent. Then the logical expression can be developed from this list.

Problem: A large department store has its own charge card. The policy for a customer to charge an item is that the customer must have a valid charge card and either a balance of less than \$500 or a charge of less than \$50.

1. The first step is to list the items the charge is dependent on and their data types:
 - a. the charge card (logical data type)
 - b. the balance (numeric data type)
 - c. the charge amount (numeric data type)
2. The second step is to write down these items as variables along with the conditions on each. Put them on one line, leaving space between, so that the logical operator can be added next. The resultant or the data type of each expression must be logical. Remember that the operators must match the data types of the operands (see Table 2.7). Do not put the logical operators in as of yet.

ChargeCard Balance < 500 Amount < 50

3. The third step is to put in the logical operators and parentheses:

ChargeCard AND (Balance < 500 OR Amount < 50)

Because the condition between the balance and the amount is one or the other, the OR operator is needed. The condition between the charge card and the other two items is both; this calls for the AND operator. Since the AND has precedence over the OR, the parentheses are added around the OR operation.

4. You have now created the logical expression. This expression can be used in various ways:
 - a. In an assignment statement:

OkToChg = ChargeCard AND (Balance < 500 OR Amount < 50)

- b. In a decision statement (this will be explained in Chapter 6):

If ChargeCard AND (Balance < 500 OR Amount < 50)

Then Print "Okay to charge"

Or Else Print "Not Okay to charge"

Summary

This chapter explains fundamental concepts for solving problems on the computer, including constants, variables, data types, common functions, operators, expressions, and equations. It also shows the beginning programmer how to set up computer expressions and equations.

Constants are values that never change during processing, even when they are named constants; variables are values that do change. Constants and variables are assigned data

types and are used in expressions and equations. The most common data types include numeric, character, and logical.

Expressions and equations use functions to process standard tasks. Most functions are part of the computer language. Expressions and equations use operators to process the data. These operators are of three types: mathematical, relational, and logical. By using operators and the hierarchy of operations, a programmer can design calculations and decisions, which are the key to solving problems on the computer.

New Terms

character data	file	mathematical operator
concatenation	function	numeric data
constant	hierarchy	operator
data	identifier	parameter
data type	information	relational operator
equation	logical data	variable
expression	logical operator	

Questions

1. Why is it important to maintain a naming convention within the company that is developing computer programs?
2. Name the rules for naming constants and variables.
3. Is *Height of students in a class* a variable or a constant?
4. Fill in the following table with the variable name and data type needed to solve an inventory problem for the University of Bristol.

Data Item	Variable Name	Data Type
a. Name of University		
b. Name of Student		
c. Application Number		
d. Number of courses applied for		
e. Fees		
f. Residential address		
g. Phone number		
h. Graduate student (Yes/No)		
i. Zip code		

5. Name the data type for each of the following constants. Explain your answer.
 - a. 5.38
 - b. "87654"

- c. *“True”*
 - d. *“A”*
 - e. 05309
 - f. *“New York”*
 - g. -389
 - h. 2.45E6
 - i. *True*
 - j. *False*
6. What is a function?
 7. Why are operators necessary?
 8. What is the difference between the operators /, \, and MOD? Give an example of each.
 9. Find the result of the following operations:
 - a. $5 + 4$
 - b. $10/2$
 - c. *True OR False*
 - d. $20 \text{ MOD } 3$
 - e. $5 < 8$
 - f. $25 \text{ MOD } 70$
 - g. *“A” > “H”*
 - h. *NOT True*
 - i. $25 \backslash 70$
 - j. *False AND True*
 - k. $20 * 0.5$
 - l. $35 \leq 35$
 - m. $35/7$
 - n. *False OR False*
 - o. *True AND True*
 - p. $50 \text{ MOD } 5$
 - q. $-35 < 67$
 - r. $4.0 \wedge 3$
 - s. $60 \backslash 9$
 - t. $35 < 35$
 - u. *True AND False*
 10. Using the hierarchy chart (Table 2.8), list the order in which the following operations would be processed. (Remember: Operations are processed left to right within a level in the hierarchy table.)
 - a. +, -, *
 - b. /, \, =
 - c. OR, *, <
 - d. NOT, AND, *
 - e. NOT, >, +
 - f. AND, OR, NOT
 - g. <, AND, >, +
 - h. *, ^, +
 - i. NOT, +, \
 - j. MOD, \, <
 11. Name the data type of the operands and the resultant of each of the following expressions or equations.

	Data Types of Operands	Data Types of Resultant
a. $A * B$		
b. $D > R$		
c. NOT C		
d. $B \text{ AND } F$		
e. $G = B$		

12. What is the difference between an expression and an equation?
13. What does the equal sign mean in an equation? Is the meaning of the equal sign as a relational operator different from that in an assignment statement? If so, in what way?

Problems

1. Evaluate the following equations, given the values $A = 18$, $B = 6$, $C = 2$, $D = 3$:
 - a. $F = A + B/C - D^2$
 - b. $F = (A + B)/C - D^2$
 - c. $F = A + B/(C - D^2)$
 - d. $F = (A + B) \text{ MOD } C$
 - e. $F = (A + B) * D^2$
2. Write the following equations in computer form for mathematical assignment:
 - a. $X = 2Y + 3Z - (Y + Z)/2YZ$
 - b. $Z^2 = X^2 + Y^2$
 - c. $(X - Y)^2 = X + Y/(2Z + 3)$
3. Are the following cases examples of equations or expressions? Justify your answer.
 - a. $A \leq B + 3$
 - b. $X - 3 = Y + Z/3$
4. Set up an equation to calculate the following (create your own variable names):
 - a. The area of a room.
 - b. The wall area of a room including windows and doors.
 - c. The wall area of a room not including two windows and a door.
 - d. The number of miles given a number of feet. (Use 5,280 feet per mile.)
 - e. The percent increase (or decrease) of a value given the beginning number and the ending number. How would the result differ between increase and decrease?
 - f. The average of five numbers.
 - g. The sale price of an item given an original price and a percentage discount.
5. Evaluate the following equations given $A = 5$, $B = 4$, $C = 3$, $D = 12$:
 - a. $E = A * B + D/C$
 - b. $E = D \text{ MOD } A * B$

- c. $E = 5 * A \setminus D * (B + 1)$
- d. $E = D / B * ((A + 4) \setminus (C + 1))$

6. Given a point (x,y) on the circumference, you want to calculate radius 'r' of a circle centered at origin. Set up an equation for this. Can you now develop a function which calculates the radius, given such a point? (Note: Recall from high school mathematics that for a point (x,y) on the circumference of a circle centered at origin, $r^2 = x^2 + y^2$).
7. Steve Thompson bought and charged an electric saw on *Date1* and made the first payment on *Date2*. Write an equation to calculate how many days (*Days*) elapsed between the two dates.

Days =

8. Eureka Lumber Company gives a 2% discount if the remaining balance is paid within 10 days of purchase. Write a logical expression to verify whether a customer qualifies for this discount. Use the following variable names:
BuyingDate: date the customer purchased the merchandise
PaidDate: date the customer made the final payment
9. A part-time employee worked 21 hours in the first week and 28 hours in the second week of a two-week pay period. He is paid a weekly salary based on a 35-hour week. What is his full-time equivalent for the two weeks based on a 40-hour week (i.e., what percentage of full time did he work)? Write a general equation that could be used to express and store the full-time equivalent of any hours worked per week.
10. Roger would like to know the average of his test scores. Write an equation that would calculate the average given five test scores. Write the equation with and without using a function.
11. Sharon is traveling from city P to city Q. The distance between the two cities is a variable because she would like to use the equation for other cities. She knows that she will be travelling half the distance at a speed of 40 miles per hour, and that the remaining half will be travelled at 55 miles per hour. Write an equation that will calculate the time it will take her to travel from one city to the next.
12. Evaluate the following equations, given $A = \text{False}$, $B = \text{True}$, $C = \text{False}$, $D = \text{True}$. (Include the structure of the order of processing—see page 44 for example.)
 - a. $R = A \text{ AND } B \text{ OR } C \text{ AND } D$
 - b. $R = \text{NOT } (A \text{ AND } B) \text{ OR } \text{NOT } (D \text{ AND } C)$
 - c. $R = (A \text{ OR } B) \text{ AND } (D \text{ OR } C)$
 - d. $R = \text{NOT } (A \text{ AND } B \text{ OR } C) \text{ AND } (A \text{ OR } B \text{ AND } D)$
 - e. $R = C \text{ OR } \text{NOT } (A \text{ AND } D) \text{ AND } (A \text{ OR } B) \text{ OR } \text{NOT } (A \text{ OR } C)$
13. Create a table that gives all possible answers for the following logical equations. (Include the structure of the order of processing—see page 45 for example.) Make clear how you set up the table.
 - a. $R = A \text{ OR } B$
 - b. $R = \text{NOT } A \text{ OR } B$
 - c. $R = A \text{ AND } B \text{ AND } (B \text{ OR } C)$
 - d. $R = \text{NOT } (A \text{ OR } B) \text{ AND } \text{NOT } (B \text{ OR } C)$
 - e. $R = B \text{ AND } \text{NOT } (A \text{ OR } C) \text{ OR } \text{NOT } (B \text{ AND } C)$

14. Evaluate the following for the values $A = 5$, $B = 2$, $C = \text{True}$, $D = \text{False}$. (Include the structure of the order of processing.)
 - a. $R = A + 3 > B - 1 \text{ AND } C \text{ OR } D$
 - b. $R = \text{NOT } C \text{ OR } D \text{ OR } A - 3 \leq B$
15. Set up a logical expression for the following conditions. A company gives a bonus at the end of each fiscal year. For an employee to get a bonus, the following must be true:
 - a. The employee has been working at the company for more than six months with no negative reports.
 - b. The employee has earned more than \$5,000 during the fiscal year.
16. Set up a logical expression for the following conditions. A retail store has this check-cashing policy:
 - a. The customer must have a driver's license.
 - b. When the check is for more than \$50, the customer must have a check-cashing card on file.

WHAT'S WRONG WITH THIS?

1. What is wrong with these variable names? Can you correct them?
 - a. *City Name* referencing the name of a city.
 - b. *Client-name* referencing a client name.
 - c. *City/State* referencing a city and state.
 - d. *LN* referencing a last name.
 - e. *Street address*
 - f. *Q* for a quantity of books
 - g. *Street_Address_for_Joe's_Hardware_Supply_Incorporated_Client*
2. The answers given are wrong. What is the correct answer and why are the answers wrong?

The answer to:	is:
a. $5 \text{ MOD } 255$	0
b. $250 \setminus 100$	2.5
3. What is wrong with the following expressions? Can you correct them?
 - a. $"L" < 5$
 - b. $"D" * "5"$
 - c. $\text{True} < "F"$
 - d. $\text{False} \geq 5$
 - e. $5.9 \setminus 2$
 - f. $3 \text{ mod } 5.2$
 - g. As an equation: $5 = \text{TotalTime} - 40$

This page intentionally left blank